TECNOLOGÍAS WEB

Curso 2025/2026

Grado en Ingeniería Informática en Tecnologías de la Información

Doble Grado Telecomunicaciones-Informática

Librería JSTL en JSP

Este documento presenta, con orientación docente y progresión gradual, el uso de las principales etiquetas de la biblioteca JSTL core y de las funciones de JSTL functions en páginas JSP. La secuencia parte de operaciones elementales de salida y asignación de variables, continúa con estructuras condicionales y de repetición, y culmina con el tratamiento de cadenas y colecciones mediante funciones auxiliares. En cada apartado se ofrece primero una explicación general y, a continuación, un ejemplo genérico seguido de su aplicación al contexto de una aplicación web de restaurantes.

La finalidad de este material es que el alumnado comprenda que la sintaxis de JSTL es estable con independencia del dominio de aplicación. Así, una vez entendida la lógica de uso sobre ejemplos neutros, puede trasladarse sin dificultad a casos prácticos como el listado, filtrado y representación de restaurantes, categorías, direcciones o descripciones.

1. Salida básica de información con c:out

La etiqueta c:out permite mostrar un valor en la página JSP de forma explícita. Es un buen punto de partida porque introduce al estudiante en la idea de que la vista puede representar datos enviados desde el servlet sin recurrir a código Java incrustado.

Ejemplo genérico

```
<c:out value="${usuario.nombre}" />
```

Aplicación a restaurantes

```
<c:out value="${restaurant.name}" />
```

En el ejemplo genérico se muestra el nombre de un usuario. En la aplicación de restaurantes, la misma idea sirve para representar el nombre del establecimiento recuperado desde el controlador.

2. Asignación de variables con c:set

La etiqueta c:set se utiliza para crear o modificar variables dentro de la propia JSP. Didácticamente resulta útil para generar títulos, mensajes de apoyo o valores temporales que luego se reutilizan en la presentación.

Ejemplo genérico

```
<c:set var="titulo" value="Panel principal" />
<h1>${titulo}</h1>
```

Aplicación a restaurantes

```
<c:set var="tituloPagina" value="Listado de restaurantes" />
<h1>${tituloPagina}</h1>
```

El alumnado observa así que la vista no solo imprime información previa, sino que también puede organizar el contenido mediante pequeñas variables de apoyo.

3. Eliminación de variables con c:remove

La etiqueta c:remove elimina una variable previamente creada. Aunque su uso no es tan frecuente como el de otras etiquetas, es adecuada para mostrar cómo se gestiona el alcance de la información dentro de la vista.

Ejemplo genérico

```
<c:set var="mensaje" value="Bienvenido al sistema" />
<p>${mensaje}</p>
<c:remove var="mensaje" />
```

Aplicación a restaurantes

```
<c:set var="aviso" value="Promoción activa hoy" />
<p>${aviso}</p>
<c:remove var="aviso" />
```

En el ámbito de restaurantes puede emplearse, por ejemplo, para retirar un aviso temporal una vez mostrado al usuario.

4. Condiciones simples con c:if

La etiqueta c:if permite ejecutar un fragmento de la vista solo cuando se cumple una condición. Su aprendizaje es esencial para que el estudiante entienda cómo personalizar la salida en función de los datos.

Ejemplo genérico

```
<c:if test="${edad >= 18}">
    <p>Usuario mayor de edad</p>
</c:if>
```

Aplicación a restaurantes

```
<c:if test="${restaurant.bikeFriendly}">
    <p>Este restaurante es Bike Friendly.</p>
</c:if>
```

La lógica es idéntica en ambos casos: una condición booleana decide si determinado contenido aparece o no en la página.

5. Decisiones múltiples con c:choose, c:when y c:otherwise

Cuando existen varios escenarios posibles, JSTL ofrece una estructura equivalente a if, else if y else. Es especialmente recomendable para mostrar clasificaciones o estados alternativos.

Ejemplo genérico

```
<c:choose>
    <c:when test="${nota >= 9}">
        <p>Calificación: sobresaliente</p>
    </c:when>
    <c:when test="${nota >= 5}">
        <p>Calificación: aprobado</p>
    </c:when>
    <c:otherwise>
        <p>Calificación: suspenso</p>
    </c:otherwise>
</c:choose>
```

Aplicación a restaurantes

```
<c:choose>
    <c:when test="${restaurant.priceRange == "alto"}">
        <p>Restaurante de gama alta</p>
    </c:when>
    <c:when test="${restaurant.priceRange == "medio"}">
        <p>Restaurante de precio medio</p>
    </c:when>
    <c:otherwise>
        <p>Restaurante económico</p>
    </c:otherwise>
</c:choose>
```

El estudiante aprende aquí a modelar distintas salidas según el valor de una variable, tanto en ejemplos abstractos como en situaciones cercanas a una interfaz real.

6. Recorrido de colecciones con c:forEach

La etiqueta c:forEach constituye uno de los recursos más importantes de JSTL, ya que permite iterar sobre listas y colecciones procedentes del servlet. Su dominio es fundamental para construir listados dinámicos.

Ejemplo genérico

```
<c:forEach var="alumno" items="${alumnos}">
    <p>${alumno.nombre}</p>
</c:forEach>
```

Aplicación a restaurantes

```
<c:forEach var="r" items="${restaurants}">
    <h2>${r.name}</h2>
    <p>${r.address}</p>
</c:forEach>
```

La traslación al dominio de restaurantes es inmediata: un listado de establecimientos no es más que una colección recorrida y representada de forma iterativa.

7. Información adicional del bucle con varStatus

El atributo varStatus de c:forEach añade información complementaria sobre el estado del recorrido, como el contador actual. Esto permite numerar resultados o detectar posiciones dentro del listado.

Ejemplo genérico

```
<c:forEach var="alumno" items="${alumnos}" varStatus="estado">
    <p>${estado.count}. ${alumno.nombre}</p>
</c:forEach>
```

Aplicación a restaurantes

```
<c:forEach var="r" items="${restaurants}" varStatus="estado">
    <p>${estado.count}. ${r.name}</p>
</c:forEach>
```

Desde el punto de vista didáctico, este paso amplía el concepto de iteración y muestra que un bucle puede ofrecer más información que el simple elemento recorrido.

8. Iteración por rango numérico con c:forEach

Además de recorrer colecciones, c:forEach puede iterar sobre un rango numérico. Este uso es especialmente útil para representar repeticiones fijas o escalas visuales.

Ejemplo genérico

```
<c:forEach var="i" begin="1" end="5">
    <p>Elemento ${i}</p>
</c:forEach>
```

Aplicación a restaurantes

```
<c:forEach var="i" begin="1" end="5">
    <span>⭐</span>
</c:forEach>
```

En una aplicación de restaurantes, este patrón puede reutilizarse para mostrar una valoración máxima, iconos o cualquier elemento repetitivo con un número prefijado de veces.

9. División de texto en tokens con c:forTokens

La etiqueta c:forTokens separa una cadena en fragmentos según uno o varios delimitadores y recorre cada parte obtenida. Es una forma sencilla de trabajar con información textual compactada.

Ejemplo genérico

```
<c:forTokens items="rojo,verde,azul" delims="," var="color">
    <p>${color}</p>
</c:forTokens>
```

Aplicación a restaurantes

```
<c:forTokens items="${restaurant.tags}" delims="," var="tag">
    <span>${tag}</span>
</c:forTokens>
```

En el dominio de restaurantes resulta útil para mostrar etiquetas como italiano, familiar o vegano cuando se almacenan en una única cadena.

10. Control básico de errores con c:catch

La etiqueta c:catch permite capturar excepciones producidas durante la evaluación de un bloque. Aunque no siempre se enseña al principio, es interesante en niveles intermedios para introducir robustez en la vista.

Ejemplo genérico

```
<c:catch var="error">
    <p>${datos.inexistente.propiedad}</p>
</c:catch>
<c:if test="${not empty error}">
    <p>Se ha producido un error al mostrar los datos.</p>
</c:if>
```

Aplicación a restaurantes

```
<c:catch var="errorRest">
    <p>${restaurant.owner.name}</p>
</c:catch>
<c:if test="${not empty errorRest}">
    <p>No se ha podido recuperar la información del propietario del restaurante.</p>
</c:if>
```

Este recurso permite ilustrar que la capa de presentación también puede prever fallos de acceso a datos anidados o inexistentes.

11. Funciones fn: para cadenas y colecciones

Una vez comprendidas las etiquetas de control básicas, el siguiente paso consiste en trabajar con las funciones de JSTL. Estas funciones se invocan dentro de expresiones EL y permiten manipular texto, calcular longitudes, buscar fragmentos o transformar cadenas. Su valor didáctico es muy alto porque complementan las estructuras anteriores y permiten enriquecer la presentación de datos en la JSP.

11.1. fn:length()

Devuelve la longitud de una cadena o el número de elementos de una colección.

Ejemplo genérico

```
<p>Total de alumnos: ${fn:length(alumnos)}</p>
```

Aplicación a restaurantes

```
<p>Total de restaurantes: ${fn:length(restaurants)}</p>
```

11.2. fn:contains()

Comprueba si una cadena contiene otra subcadena.

Ejemplo genérico

```
<c:if test="${fn:contains(correo, "@gmail.com")}">
    <p>Correo de Gmail detectado</p>
</c:if>
```

Aplicación a restaurantes

```
<c:if test="${fn:contains(restaurant.name, "Pizza")}">
    <p>Este restaurante parece especializado en pizza.</p>
</c:if>
```

11.3. fn:containsIgnoreCase()

Realiza la misma comprobación sin distinguir mayúsculas y minúsculas.

Ejemplo genérico

```
<c:if test="${fn:containsIgnoreCase(nombreCurso, "java")}">
    <p>Curso relacionado con Java</p>
</c:if>
```

Aplicación a restaurantes

```
<c:if test="${fn:containsIgnoreCase(restaurant.description, "vegano")}">
    <p>Incluye opciones veganas.</p>
</c:if>
```

11.4. fn:startsWith()

Permite verificar si una cadena comienza por un determinado prefijo.

Ejemplo genérico

```
<c:if test="${fn:startsWith(codigo, "INF")}">
    <p>Asignatura del área de informática</p>
</c:if>
```

Aplicación a restaurantes

```
<c:if test="${fn:startsWith(restaurant.name, "La ")}">
    <p>Nombre comercial iniciado por artículo determinado.</p>
</c:if>
```

11.5. fn:endsWith()

Permite verificar si una cadena termina con un determinado sufijo.

Ejemplo genérico

```
<c:if test="${fn:endsWith(documento, ".pdf")}">
    <p>El archivo es un PDF</p>
</c:if>
```

Aplicación a restaurantes

```
<c:if test="${fn:endsWith(restaurant.website, ".com")}">
    <p>La web del restaurante usa dominio .com</p>
</c:if>
```

11.6. fn:indexOf()

Devuelve la posición de una subcadena dentro de otra.

Ejemplo genérico

```
<p>Posición de "admin": ${fn:indexOf(rolUsuario, "admin")}</p>
```

Aplicación a restaurantes

```
<p>Posición de "bar" en el nombre: ${fn:indexOf(restaurant.name, "bar")}</p>
```

11.7. fn:substring()

Extrae una parte de una cadena según una posición inicial y final.

Ejemplo genérico

```
<p>${fn:substring(descripcion, 0, 20)}...</p>
```

Aplicación a restaurantes

```
<p>${fn:substring(restaurant.description, 0, 40)}...</p>
```

11.8. fn:substringBefore() y fn:substringAfter()

Permiten obtener la parte anterior o posterior a un separador.

Ejemplo genérico

```
<p>Usuario: ${fn:substringBefore(correo, "@")}</p>
<p>Dominio: ${fn:substringAfter(correo, "@")}</p>
```

Aplicación a restaurantes

```
<p>Dominio web: ${fn:substringAfter(restaurant.website, "www.")}</p>
```

11.9. fn:toUpperCase() y fn:toLowerCase()

Transforman el contenido textual a mayúsculas o minúsculas.

Ejemplo genérico

```
<p>${fn:toUpperCase(usuario.nombre)}</p>
<p>${fn:toLowerCase(usuario.nombre)}</p>
```

Aplicación a restaurantes

```
<p>${fn:toUpperCase(restaurant.name)}</p>
<p>${fn:toLowerCase(restaurant.category)}</p>
```

11.10. fn:trim()

Elimina espacios innecesarios al principio y al final de la cadena.

Ejemplo genérico

```
<p>${fn:trim(textoIntroducido)}</p>
```

Aplicación a restaurantes

```
<p>${fn:trim(restaurant.address)}</p>
```

11.11. fn:replace()

Sustituye un fragmento de texto por otro.

Ejemplo genérico

```
<p>${fn:replace(frase, "Hola", "Bienvenido")}</p>
```

Aplicación a restaurantes

```
<p>${fn:replace(restaurant.description, "barato", "económico")}</p>
```

11.12. fn:split()

Divide una cadena en varias partes y permite tratarlas como una colección.

Ejemplo genérico

```
<c:forEach var="palabra" items="${fn:split("uno,dos,tres", ",")}">
    <p>${palabra}</p>
</c:forEach>
```

Aplicación a restaurantes

```
<c:forEach var="categoria" items="${fn:split(restaurant.categories, ",")}">
    <span>${categoria}</span>
</c:forEach>
```

11.13. fn:join()

Une los elementos de una colección en una sola cadena mediante un separador.

Ejemplo genérico

```
<p>${fn:join(listaRoles, " - ")}</p>
```

Aplicación a restaurantes

```
<p>${fn:join(restaurantCategoryNames, ", ")}</p>
```

11.14. fn:escapeXml()

Escapa caracteres especiales para que no se interpreten como marcado HTML o XML.

Ejemplo genérico

```
<p>${fn:escapeXml(textoUsuario)}</p>
```

Aplicación a restaurantes

```
<p>${fn:escapeXml(restaurant.description)}</p>
```

12. Integración progresiva: ejemplo completo aplicado a restaurantes

Después de trabajar cada recurso por separado, conviene presentar al alumnado una JSP donde varias etiquetas y funciones aparezcan integradas en una sola solución. De este modo se aprecia la cohesión entre asignación de variables, decisiones, iteraciones y tratamiento de cadenas.

```
<%@ taglib prefix="c" uri="http://java.sun.com/jsp/jstl/core" %>
<%@ taglib prefix="fn" uri="http://java.sun.com/jsp/jstl/functions" %>

<c:set var="tituloPagina" value="Listado de restaurantes" />
<h1>${tituloPagina}</h1>
<p>Total encontrados: ${fn:length(restaurants)}</p>

<c:choose>
   <c:when test="${empty restaurants}">
      <p>No hay restaurantes disponibles.</p>
   </c:when>
   <c:otherwise>
      <c:forEach var="r" items="${restaurants}" varStatus="estado">
         <div>
            <h2>${estado.count}. <c:out value="${r.name}" /></h2>
            <p>Dirección: ${fn:trim(r.address)}</p>

            <c:choose>
               <c:when test="${r.bikeFriendly}">
                  <p>Bike Friendly: Sí</p>
               </c:when>
               <c:otherwise>
                  <p>Bike Friendly: No</p>
               </c:otherwise>

            </c:choose>

            <c:if test="${fn:length(r.description) > 50}">
               <p>Descripción breve: ${fn:substring(r.description, 0, 50)}...</p>
            </c:if>

            <c:if test="${fn:containsIgnoreCase(r.description, 'vegano')}">
               <p>Incluye opciones veganas.</p>
            </c:if>
```

```
            <p>Categorías:</p>
            <c:forEach var="cat" items="${fn:split(r.categories, ',')}">
               <span>${fn:trim(cat)}</span>
            </c:forEach>

         </div>
         <hr>
      </c:forEach>
   </c:otherwise>
</c:choose>
```